

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 06 -

TESTE UNITÁRIO & TESTE INTEGRAÇÃO

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	14
REFERÊNCIAS.....	15

EMSE

O QUE VEM POR AÍ?

Não basta que o software desenvolvido resolva o problema; precisamos pensar sobre o desempenho, facilidade de entendimento dos pares e na manutenção futura. Nesta aula, vamos aprender sobre testes unitários e integrados, os quais são fundamentais no desenvolvimento de software, pois garantem a qualidade e a robustez do código.

EMANIP

HANDS ON

Para esta aula, usaremos um conteúdo básico que ficará disponível no repositório da disciplina, na branch testes: https://github.com/rocaabrera/curso_mlops/tree/testes.

Faremos a implementação dos testes em nosso pacote e todos os recursos se encontram na branch testes_empacotamento_codigo: https://github.com/rocaabrera/curso_mlops/tree/testes_empacotamento_codigo.

Já a implementação dos testes para a nossa aplicação está na branch testes_containers: https://github.com/rocaabrera/curso_mlops/tree/testes_containers.

SAIBA MAIS

Esta aula tem o objetivo de ser uma introdução ao conceito de testes e também sobre a utilização do pacote pytest para implementação de testes unitários, os quais são fundamentais no desenvolvimento de software, pois permitem verificar se pequenas partes do código, chamadas de unidades, funcionam conforme o esperado. Eles ajudam a identificar e corrigir erros precocemente, antes que se propaguem para outras partes do sistema, tornando o processo de desenvolvimento mais eficiente e confiável. Dessa forma, é possível fazer alterações ou adicionar novas funcionalidades com mais segurança, sabendo que os testes sinalizarão qualquer comportamento inesperado que possa ter sido introduzido. Isso reduz o risco de bugs e melhora a qualidade da aplicação.

Partindo de um folder sem código, vamos construir testes unitários. Primeiro, vamos criar dois diretórios: **src** e **tests**. Dentro de src vamos criar um módulo chamado main.py e no folder tests vamos criar um módulo chamado test_main.py (os quais precisam do prefixo **test**), cada um com seu respectivo código como mostra a Figura 1:



Figura 1 - Código genérico e um teste unitário
Fonte: Elaborado pelo autor (2024)

Agora vamos instalar o pacote pytest e pytest-cov, os quais permitirão realizar as operações mais comuns relacionadas aos testes. Execute o seguinte comando:

```
pip install -U pytest pytest-cov
```

Vamos executar nosso teste com o comando:

pytest tests/

E então você vai receber uma mensagem com o seguinte erro:

```
(ins)$ pytest tests/
===== test session starts =====
platform darwin -- Python 3.11.4, pytest-8.1.1, pluggy-1.4.0
rootdir: /Users/rodrigocastaldoni/Desktop/curso_mlops
plugins: anyio-4.4.0
collected 0 items / 1 error

===== ERRORS =====
ERROR collecting tests/test_main.py
ImportError while importing test module '/Users/rodrigocastaldoni/Desktop/curso_mlops/tests/test_main.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
../.pyenv/versions/3.11.4/lib/python3.11/importlib/_init_.py:126: in import_module
    return _bootstrap.gcd.import(name[level:], package, level)
tests/test_main.py:1: in <module>
    from src.main import Calculadora
E ModuleNotFoundError: No module named 'src'

===== short test summary info =====
ERROR tests/test_main.py
!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!
===== 1 error in 0.05s =====
```

Figura 2 - Erro frequente quando começamos a testar nossas aplicações
Fonte: Elaborado pelo autor (2024)

Para resolver esse erro, basta adicionar o arquivo `__init__.py` no diretório **tests** e executar o comando novamente:

```
(ins)$ pytest -s tests/
===== test session starts =====
platform darwin -- Python 3.11.4, pytest-8.1.1, pluggy-1.4.0
rootdir: /Users/rodrigocastaldoni/Desktop/curso_mlops
plugins: anyio-4.4.0
collected 1 item

tests/test_main.py .

===== 1 passed in 0.01s =====
```

Figura 3 - Execução dos testes com sucesso
Fonte: Elaborado pelo autor (2024)

Em um projeto real você vai ter vários módulos de teste. Para executar um módulo específico, basta passar o PATH até o módulo de teste no comando anterior, como por exemplo:

pytest tests/test_main.py

Além disso, é possível especificar a função dentro do módulo que precisa ser testado com o parâmetro **-k**, como por exemplo:

pytest tests/test_main.py -k test_adicao

Entretanto, nossos testes não exploram toda a nossa aplicação calculadora. Para ver isso, podemos executar o comando:

pytest tests/ --cov=./src

A Figura 4 mostra o coverage da nossa aplicação, isto é, qual é a porcentagem de linhas testadas por módulo.

```
(ins)$ pytest tests/ --cov=./src
===== test session starts =====
platform darwin -- Python 3.11.4, pytest-8.1.1, pluggy-1.4.0
rootdir: /Users/rodrigocastaldoni/Desktop/curso_mlops
plugins: cov-5.0.0, anyio-4.4.0
collected 1 item

tests/test_main.py . [100%]

----- coverage: platform darwin, python 3.11.4-final-0 -----
Name      Stmts  Miss  Cover
-----
src/main.py    14     6    57%
TOTAL         14     6    57%

===== 1 passed in 0.02s =====
```

Figura 4 - Execução do coverage da aplicação
Fonte: Elaborado pelo autor (2024)

Entretanto, esse comando não mostra quais linhas não foram testadas. Para fazer isso, vamos adicionar o parâmetro `--cov-report=html`, usando o seguinte comando:

pytest tests/ --cov=./src --cov-report=html

Ele vai gerar um diretório chamado `htmlcov` e, dentro dele, terá um arquivo chamado `index.html`. Abra esse arquivo em seu navegador browser, conforme mostra a Figura 5:

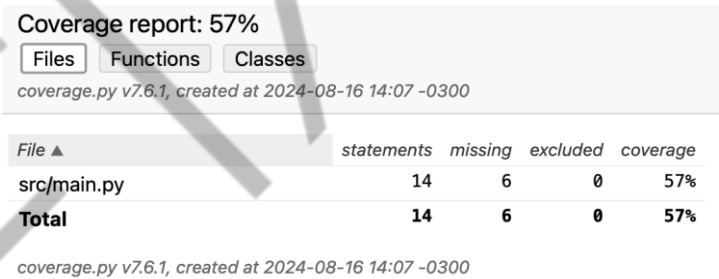


Figura 5 - Coverage mostrado no browser
Fonte: Elaborado pelo autor (2024)

Ao clicar no arquivo `src/main.py`, será carregada a página mostrada na Figura 6:

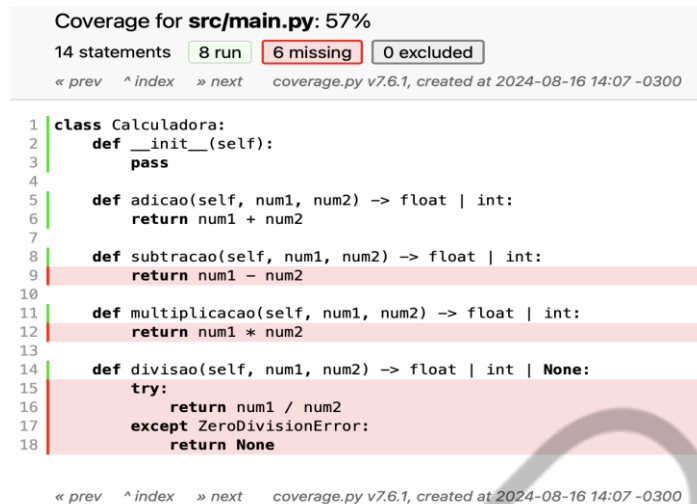


Figura 6 - Coverage por linha do módulo main
Fonte: Elaborado pelo autor (2024)

Perceba que não foram testadas as funções subtracao, multiplicacao e divisao. Vamos criar esses testes, conforme mostra a Figura 7:

```

test_main.py U x
tests > test_main.py > ...
1  from src.main import Calculadora
2
3  def test_adicao():
4      calc = Calculadora()
5      assert calc.adicao(1, 2) == 3
6
7  def test_subtracao():
8      calc = Calculadora()
9      assert calc.subtracao(5, 3) == 2
10
11  def test_multiplicacao():
12      calc = Calculadora()
13      assert calc.multiplicacao(4, 2) == 8
14
15  def test_divisao():
16      calc = Calculadora()
17      assert calc.divisao(0, 5) == 0
18      assert calc.divisao(8, 4) == 2
19
20

```

Figura 7 - Coverage por linha do módulo main
Fonte: Elaborado pelo autor (2024)

Executando o comando anterior, obtemos o coverage mostrado na Figura 8:

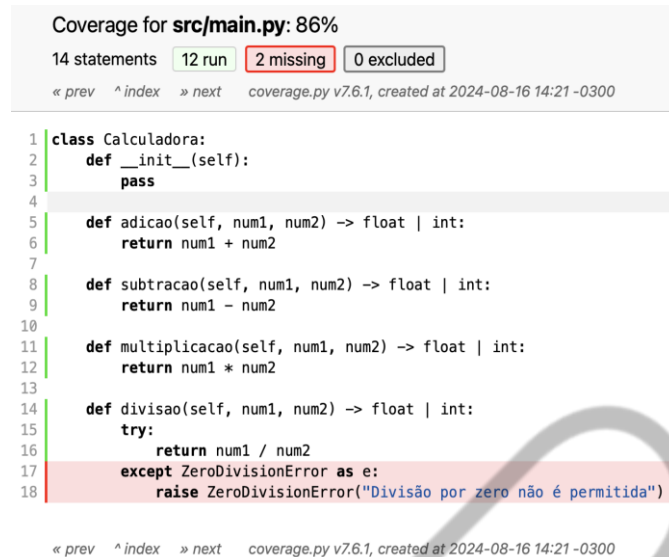


Figura 8 - Coverage por linha do módulo main depois da implementação de mais testes
Fonte: Elaborado pelo autor (2024)

A Figura 8 mostra que a linha referente ao levantamento da exceção não foi testada. Podemos fazer esse tipo de teste conforme mostra a Figura 9:

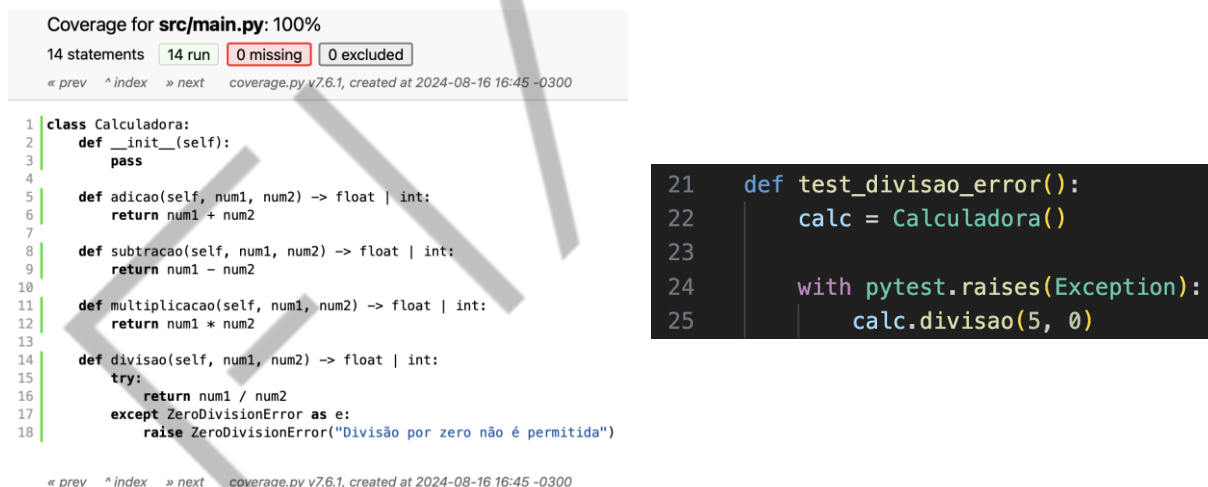
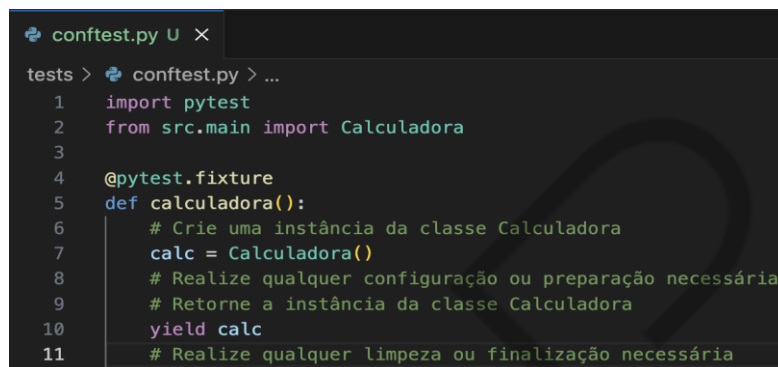


Figura 9 - Teste de levantamento de exceções
Fonte: Elaborado pelo autor (2024)

Finalmente temos 100% de coverage em nosso código. Apesar de ser bom termos um código assim, na prática é muito difícil de isso acontecer. Implemente testes que verifiquem o comportamento do seu software e, naturalmente, o coverage deve aumentar. Caso esteja difícil testar o seu código, pode ser que ele esteja complicado demais, normalmente um código bem escrito é simples de testar.

Agora vamos introduzir o arquivo **conftest.py** (o qual fica dentro do diretório **tests**). Antes da execução de testes, o módulo **conftest.py** é executado. Logo, implementamos o código que queremos compartilhar entre vários testes diferentes. Nele, usualmente, são implementadas as **fixtures**, funções com um decorador especial do pytest, conforme mostra a Figura 10:



```
conftest.py U X
tests > conftest.py > ...
1 import pytest
2 from src.main import Calculadora
3
4 @pytest.fixture
5 def calculadora():
6     # Crie uma instância da classe Calculadora
7     calc = Calculadora()
8     # Realize qualquer configuração ou preparação necessária
9     # Retorne a instância da classe Calculadora
10    yield calc
11    # Realize qualquer limpeza ou finalização necessária
```

Figura 10 - Implementação de fixture no módulo conftest
Fonte: Elaborado pelo autor (2024)

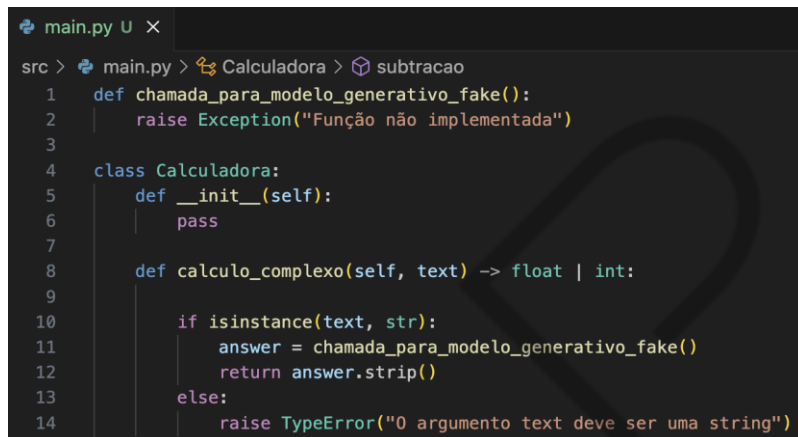
Agora podemos utilizar essa fixture em nossos testes, conforme mostra a Figura 11:



```
test_main.py U X
tests > test_main.py > ...
1 import pytest
2
3 def test_adicao(calculadora):
4     assert calculadora.adicao(1, 2) == 3
5
6 def test_subtracao(calculadora):
7     assert calculadora.subtracao(5, 3) == 2
8
9 def test_multiplicacao(calculadora):
10    assert calculadora.multiplicacao(4, 2) == 8
11
12 def test_divisao(calculadora):
13    assert calculadora.divisao(0, 5) == 0
14    assert calculadora.divisao(8, 4) == 2
15
16 def test_divisao_error(calculadora):
17    with pytest.raises(Exception):
18        calculadora.divisao(5, 0)
```

Figura 11 - Utilização da fixture nos testes
Fonte: Elaborado pelo autor (2024)

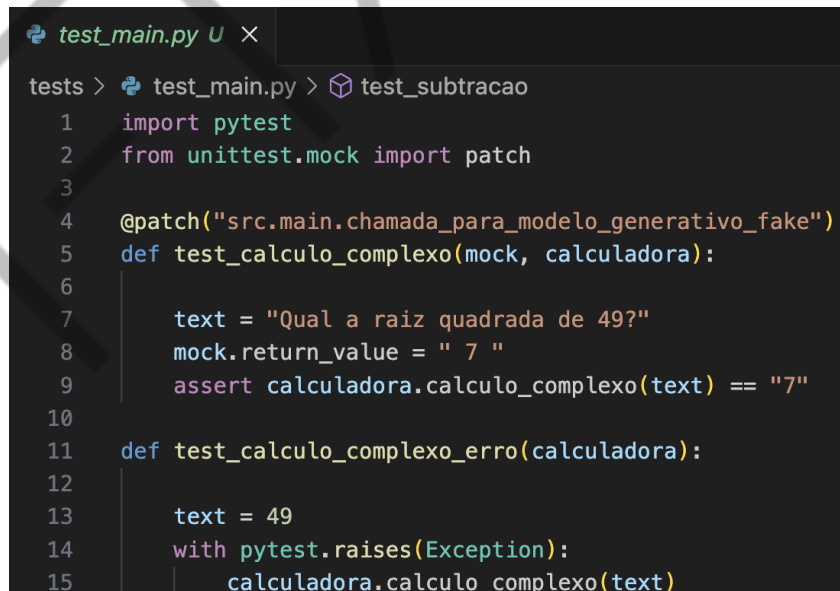
Para terminar o tema de testes unitários, vamos explicar o que é um Mock. Normalmente, existirá uma parte do seu código que você não quer verificar nos testes unitários. Imagine que nossa calculadora tenha uma função capaz de fazer cálculos muito complexos usando uma chamada para um modelo generativo, conforme mostra a Figura 12:



```
main.py U X
src > main.py > Calculadora > subtracao
1 def chamada_para_modelo_generativo_fake():
2     raise Exception("Função não implementada")
3
4 class Calculadora:
5     def __init__(self):
6         pass
7
8     def calculo_complexo(self, text) -> float | int:
9
10        if isinstance(text, str):
11            answer = chamada_para_modelo_generativo_fake()
12            return answer.strip()
13        else:
14            raise TypeError("0 argumento text deve ser uma string")
```

Figura 12 - Implementação da função que faz chamada generativa
Fonte: Elaborado pelo autor (2024)

Entretanto, a saída do modelo generativo não é determinística, logo deve ser feito o Mock dessa função, conforme mostra a Figura 13:



```
test_main.py U X
tests > test_main.py > test_subtracao
1 import pytest
2 from unittest.mock import patch
3
4 @patch("src.main.chamada_para_modelo_generativo_fake")
5 def test_calculo_complexo(mock, calculadora):
6
7     text = "Qual a raiz quadrada de 49?"
8     mock.return_value = " 7 "
9     assert calculadora.calculo_complexo(text) == "7"
10
11 def test_calculo_complexo_erro(calculadora):
12
13     text = 49
14     with pytest.raises(Exception):
15         calculadora.calculo_complexo(text)
```

Figura 13 - Implementação do mock para teste de função generativa
Fonte: Elaborado pelo autor (2024)

A Figura 14 mostra o coverage após a implementação da função generativa.



Figura 14 - Coverage após implementação da função generativa
Fonte: Elaborado pelo autor (2024)

Entretanto, dessa vez, não queremos testar essa função. No teste, trocamos a função chamada_para_modelo_generativo_fake do arquivo src.main por um mock .

Além disso, especificamos que, ao tentar executar a função chamada_para_modelo_generativo_fake, seja retornada a string ' 7 '. Porém, o resultado esperado é '7', pois aplicamos o método **.strip()** à saída da função generativa. A implementação de mocks pode ser uma das etapas mais complexas na criação de testes unitários, por isso **recomendamos fortemente assistir a videoaula para aprofundar seus conhecimentos sobre o assunto.**

Vale lembrar que existem várias outras features disponíveis no pytest, como, por exemplo, o decorador parametrize, que permite simplificar o teste implementado, conforme mostra a Figura 15:

```

26 @pytest.mark.parametrize("entrada, saida", [((0,5), 0), ((8,4), 2)])
27 def test_divisao(entrada, saida, calculadora):
28     x, y = entrada
29     assert calculadora.divisao(x,y) == saida

```

Figura 15 - Coverage após implementação da função generativa
Fonte: Elaborado pelo autor (2024)

Para finalizar esta aula, vamos comentar brevemente sobre o que são testes de integração. Eles são implementados da mesma forma que os testes unitários, mas devemos evitar os mock das comunicações entre os componentes, pois os testes de integração são implementados para testar exatamente essas comunicações. Resumindo, o que diferencia um teste unitário de um teste de integração vai depender de como você entende a sua aplicação.

EMSE

O QUE VOCÊ VIU NESTA AULA?

Nesta aula você aprendeu como implementar testes unitários em Python usando o pacote pytest. Além disso, aprendeu sobre cobertura da aplicação e como realizar mocks.

EMSE

REFERÊNCIAS

PEPS. **Storing project metadata in pyproject.toml**. 2024. Disponível em: <<https://peps.python.org/pep-0621/>>. Acesso em: 10 set. 2024.

PYTEST. **Full pytest documentation**. 2024. Disponível em: <<https://docs.pytest.org/en/7.1.x/contents.html>>. Acesso em: 10 set. 2024.

STRAUB, B.; CHACON, S. **Pro Git**. São Paulo: APRESS, 2024.

PALAVRAS-CHAVE

Palavras-chave: Testes. Qualidade. Cobertura.

EMSE



POSTECH